

Цель работы

Ознакомиться с методами и средствами построения отказоустойчивых решений на базе СУБД Postgres; получить практические навыки восстановления работы системы после отказа.

Требования к выполнению работы

- В качестве хостов использовать одинаковые виртуальные машины.
- В первую очередь необходимо обеспечить сетевую связность между ВМ.
- Для подключения к СУБД, например через `psql`, использовать отдельную виртуальную или физическую машину.
- Демонстрировать наполнение базы и доступ на запись на примере не менее чем двух таблиц, столбцов, строк, транзакций и клиентских сессий.

Этапы выполнения работы

Этап 1. Конфигурация

Развернуть Postgres на двух узлах в режиме горячего резерва: Master + Hot Standby. Не использовать дополнительные пакеты. Продемонстрировать доступ в режиме чтения и записи на основном сервере, доступ в режиме чтения на резервном сервере, а также актуальность данных на нём.

Для развёртывания нескольких узлов Postgres и обеспечения сетевой связности использовался *docker compose*.

docker-compose.yml

Два контейнера находятся в общей сети `pg-net`.

```
services:
  pg-1:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret123
    ports:
      - "5432:5432"
    volumes:
      - ./pgdata/pg1:/var/lib/postgresql/data
    networks:
      - pg-net

  pg-2:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret123
    ports:
      - "5433:5432"
    volumes:
```

```

- ./pgdata/pg2:/var/lib/postgresql/data
networks:
- pg-net

networks:
pg-net:
driver: bridge
name: pg-net

```

Шаги для режима Hot Standby

pg-1 — основной узел.

pg-2 — резервный узел.

На pg-1 в файл `postgresql.conf` были добавлены следующие параметры:

```

listen_addresses = '*'
max_connections = 100
shared_buffers = 128MB
dynamic_shared_memory_type = posix

# hot-standby setup
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = 256MB
hot_standby = on

max_wal_size = 1GB
min_wal_size = 80MB
log_timezone = 'Etc/UTC'
datestyle = 'iso, mdy'
timezone = 'Etc/UTC'
lc_messages = 'en_US.utf8'
lc_monetary = 'en_US.utf8'
lc_numeric = 'en_US.utf8'
lc_time = 'en_US.utf8'
default_text_search_config = 'pg_catalog.english'

```

В файл `pg_hba.conf` были добавлены правила доступа:

```

host replication replicator 0.0.0.0/0 md5
host all all 0.0.0.0/0 md5

```

На pg-1 был создан пользователь для репликации:

```

CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'replpass';

```

Создать backup pg-1 и положить его в pg-2

```
docker run --rm \
  --network pg-net \
  -v "$PWD/pgdata/pg2:/var/lib/postgresql/data" \
  postgres:16 \
  bash -c '
    export PGPASSWORD=replpass
    pg_basebackup \
      -h rshd-lab-3-pg-1-1 \
      -p 5432 \
      -U replicator \
      -D /var/lib/postgresql/data \
      -Fp \
      -Xs \
      -P
  '
```

На pg-2 добавить в postgresql.conf

```
hot_standby = on
primary_conninfo = 'host=rshd-lab-3-pg-1-1 port=5432 user=replicator
password=replpass application_name=pg2'
```

На pg-2 создать standby signal

```
touch ./pgdata/pg2/standby.signal
sudo chown -R 999:999 ./pgdata/pg2
```

Проверка репликации

Проверка pg_stat_replication

```
postgres=# SELECT application_name, state, sync_state, client_addr
FROM pg_stat_replication;
 application_name | state | sync_state | client_addr
-----+-----+-----+-----
pg2               | streaming | async      | 172.21.0.2
(1 row)
```

На основном сервере была создана тестовая таблица:

```
CREATE TABLE repl_test(
  id serial primary key,
  msg text
);

INSERT INTO repl_test(msg)
VALUES ('hello from master');
```

На резервном сервере была выполнена проверка:

```
SELECT * FROM repl_test;
```

```
id |      msg
----+-----
 1 | hello from master
```

Демонстрационные данные

Для демонстрации записи и репликации использовались две таблицы (users и orders), наполняемые через утилиту db-app (Go, pgx). Миграция:

```
CREATE TABLE users (
    user_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    name     varchar(100) not null,
    email    varchar(50)  not null,
    address  varchar(100) not null,
    password varchar(100) unique not null
);

CREATE TABLE orders (
    id          bigint generated always as identity primary key,
    user_id     uuid references users(user_id) on delete cascade,
    sku         bigint,
    price       double precision
);
```

Наполнение и проверка:

```
# наполняем pg-1 (master)
./bin/db-app -pg=1 seed 100

# читаем с pg-1 (порт 5432) и pg-2 (порт 5433)
psql -h 127.0.0.1 -p 5432 -U postgres -d mydb -c "SELECT count(*) FROM users;"
psql -h 127.0.0.1 -p 5433 -U postgres -d mydb -c "SELECT count(*) FROM users;"
```

Обе команды возвращают одно и то же количество строк — данные на резервном сервере актуальны. Попытка записи на pg-2 ожидаемо отбрасывается:

```
ERROR: cannot execute INSERT in a read-only transaction
```

Этап 2. Симуляция и обработка сбоя

2.1 Подготовка: клиентские сессии

Для демонстрации работы нескольких клиентов параллельно был подготовлен скрипт scripts/02_sessions.sh, поднимающий 4 фоновые сессии:

- сессия 1 — циклический INSERT в orders на pg-1 (writer);
- сессия 2 — INSERT в users в явной транзакции BEGIN/COMMIT на pg-1 (writer);
- сессия 3 — SELECT count(*) на pg-1 (reader);
- сессия 4 — SELECT count(*) на pg-2 (reader, hot standby).

Логи каждой сессии сохраняются в `scripts/sessions/*.log`. В нормальном состоянии writer'ы успешно фиксируют транзакции, reader на pg-2 видит обновления с минимальной задержкой репликации.

2.2 Сбой: переполнение раздела с PGDATA

Сам Docker volume на 473 GB не подходит для эмуляции — забить весь хостовый диск нельзя. Поэтому был сделан ограниченный раздел: образ диска 500 MB, отформатированный в ext4 и смонтированный через loop в каталог `./pgdata/pg1-mnt`. После переноса PGDATA volume pg-1 в `docker-compose.yml` был перенаправлен на этот mount (`scripts/01_setup_limited_volume.sh`).

Заполнение раздела (`scripts/03_fill_disk.sh`) выполняется изнутри контейнера:

```
docker exec rshd-lab-3-pg-1-1 bash -c '
i=0
while dd if=/dev/zero of=/var/lib/postgresql/data/garbage_${i}.bin \
      bs=1M count=50 2>/dev/null; do i=$((i+1)); done
'
```

После запуска `df -h /var/lib/postgresql/data` показывает `Use% = 100%`.

2.3 Обработка: логи, failover, доступ

Релевантные строки из `docker logs rshd-lab-3-pg-1-1`:

```
PANIC: could not write to file "pg_wal/xlogtemp.NN": No space left on device
LOG:   WAL writer process (PID NN) was terminated by signal 6: Aborted
LOG:   terminating any other active server processes
FATAL: the database system is in recovery mode
```

Сразу после этого `psql -p 5432` начинает возвращать ошибку соединения, writers в сессиях 1 и 2 падают.

Failover выполняется на pg-2 через встроенную функцию (без сторонних пакетов):

```
SELECT pg_promote(wait => true, wait_seconds => 30);
SELECT pg_is_in_recovery(); -- false
```

Клиенты переключаются на порт 5433 и продолжают работу. Появляется новая запись «after-failover» в users (см. `scripts/04_failover.sh`).

Этап 3. Восстановление

1. На хосте удаляются мусорные файлы из `./pgdata/pg1-mnt`, очищается каталог PGDATA pg-1 (`sudo rm -rf ./pgdata/pg1-mnt/*`).
2. На освободившийся каталог делается `pg_basebackup` с нового мастера pg-2 с опцией `-R` — postgres сам создаёт `standby.signal` и записывает `primary_conninfo`:

```
docker run --rm --network pg-net \
-v "$PWD/pgdata/pg1-mnt:/var/lib/postgresql/data" \
-e PGPASSWORD=replpass postgres:16 \
```

```
pg_basebackup -h rshd-lab-3-pg-2-1 -p 5432 -U replicator \  
-D /var/lib/postgresql/data -Fp -Xs -P -R
```

После этого pg-1 поднимается как standby, догоняет состояние pg-2 — таким образом «накапываются» все изменения, выполненные на этапе 2.3.

3. Для возврата к исходной конфигурации (pg-1 — master, pg-2 — standby) выполняется контролируемый switchover (scripts/06_switchback.sh):

- CHECKPOINT на pg-2 и ожидание совпадения pg_current_wal_lsn() на мастере и pg_last_wal_replay_lsn() на pg-1;
- остановка pg-2;
- pg_promote() на pg-1;
- пересборка pg-2 как standby через pg_basebackup с pg-1.

4. Финальная проверка:

```
-- pg-1: pg_is_in_recovery() = false  
-- pg-2: pg_is_in_recovery() = true  
-- pg_stat_replication на pg-1 показывает streaming/async к pg2
```

Клиенты возвращаются на pg-1 (порт 5432), запись и чтение работают, реплика на pg-2 актуальна.

Вопросы для подготовки к защите

Синхронная и асинхронная репликация

При **асинхронной** репликации мастер фиксирует транзакцию, не дожидаясь подтверждения от реплики: WAL отправляется в фоне. Это даёт максимальную пропускную способность, но в случае сбоя мастера часть зафиксированных транзакций (то, что ещё не успело уйти на реплику) теряется — RPO>0.

При **синхронной** репликации мастер дожидается, пока указанные в synchronous_standby_names реплики не подтвердят запись WAL (на уровне remote_write, remote_flush или remote_apply). RPO=0, но latency коммита увеличивается на сетевой round-trip и время записи на реплике; при недоступности синхронной реплики мастер «зависает» на коммитах, если не настроена кворумная схема (ANY n (...)).

Асинхронная репликация применяется для горячего резерва, read-replicas под чтение, географически удалённых стендов; синхронная — там, где недопустима потеря данных (платёжные системы, журналы аудита).

Active-Active и Active-Standby

В **Active-Standby** в каждый момент времени запись принимает только один узел (master), остальные хранят копию данных и могут обслуживать чтение (как в этой работе). Failover требует промоута standby и переключения клиентов — есть простой. Конфликтов записи нет.

В **Active-Active** запись принимают несколько узлов одновременно. Это даёт горизонтальное масштабирование записи и более простой failover, но требует решения конфликтов: одинаковые ключи, обновляемые на разных узлах, должны примиряться (last-write-wins, CRDT, ручное разрешение). В Postgres поддержка из коробки ограничена логической репли-

кацией; полноценные мульти-мастер реализации требуют дополнительных пакетов (BDR, pgEdge и т.п.), что выходит за рамки задания.

Active-Standby — стандарт для OLTP с требованиями целостности; Active-Active — для гео-распределённых систем, где допустимы eventual consistency и конфликтные правила.

Балансировка нагрузки

Балансировка нагрузки — распределение клиентских запросов между несколькими узлами по выбранной стратегии (round-robin, least-connections, по типу запроса). Для Postgres типичные применения:

- разделение read/write: запись идёт на master, чтение — на standby-узлы (через приложение, PgBouncer + HAProxy, или специализированные прокси вроде Pgpool-II);
- failover/health-checks: балансировщик исключает упавший узел из ротации;
- сглаживание пиков нагрузки на чтение, удешевление за счёт переноса аналитических запросов на реплики.

От чего зависит время простоя при отказе

- **Время обнаружения отказа** — частота health-check, таймауты соединений; чем чаще пинг, тем быстрее заметим, но больше ложных срабатываний.
- **Способ failover**: ручной (как в этой работе через `pg_promote()`) — минуты или больше; автоматический (Patroni, repmgr, etcd-based fencing) — секунды.
- **Согласованность данных на момент сбоя**: при асинхронной репликации иногда требуется решить, что делать с «отстающим» хвостом WAL — это занимает время.
- **Переключение клиентов**: статический DNS — десятки секунд-минуты; виртуальный IP — почти мгновенно; пул соединений с retry — определяется политикой клиента.
- **Восстановление упавшего узла**: размер БД, скорость диска, способ восстановления (`pg_basebackup` vs `pg_rewind`).
- **Тип отказа**: сбой ОС/процесса — быстро; полная потеря данных — нужно полное копирование с реплики.